

Technical Manual for ChainIDentity

Brandon Magaña Avalos
Student ID – 0244582
bavalosmm@gmail.com
Student of ISGC
Juan Carlos López Pimentel
clopezp@up.edu.mx
Research Professor - Faculty of Engineering

ChainIDentity: Architecture, Implementation, and Applications of a Blockchain-Based Digital Identity Ecosystem.

Abstract

This manual presents ChainIDentity a decentralized application (Dapp) that operates on a blockchain network, in particular Ganache. Designed to digitize and secure identities and physical objects, while maintaining interoperability and user privacy. ChainIDentity implements microservices architecture utilizing Ethereum smart contracts, RESTful APIs, and a hybrid blockchain-database approached to manage user authentication, digital identities, birth certificates, and scholarly credentials. The proposed solution addresses critical challenges in digital identity management including data integrity, authenticity verification, and decentralized trust establishment. Through the implementation of four core smart contracts and eight microservices, the system demonstrates scalability, security, and practical applicability across multiple platforms including local development environments and cloud infrastructure. Performance evaluation indicates successful deployment across diverse hardware configurations and operating systems, while security analysis reveals robust access control mechanisms and cryptographic verification processes. The system's modular design enables extensibility for additional identity components and integration with existing government and institutional systems.

Keywords: Blockchain, Digital Identity, Smart Contracts, Microservices, Ethereum, Identity Verification

1. System Architecture

1.1 Overall Architecture Design

ChainIDentity employs a microservices architecture pattern, decomposing the system into specialized, independently deployable services. This architectural approach enables scalability, maintainability, and technology diversity across different system components.

The system architecture consists of three primary layers:

1. **Blockchain Layer**: Ethereum-based smart contracts providing immutable identity storage and verification
2. **Application Layer**: Node.js microservices implementing business logic and API endpoints
3. **Presentation Layer**: Web-based frontend applications for user interaction

1.2 Microservices Components

As illustrated in Figure 1, ChainIDentity implements a comprehensive microservices architecture with eight core components distributed across three distinct layers, all operating within a unified Docker network infrastructure. Each microservice serves specific identity management functions:

1. **API Gateway Service:** Positioned as the central entry point in the Application Layer (Figure 1), this service acts as the primary routing and load balancing component. It manages all external API requests from the four Frontend Services and intelligently distributes them to the appropriate backend microservices, ensuring optimal resource utilization and system performance.

2. **User Management Service:** Located within the Application Layer microservices cluster (Figure 1), this service implements comprehensive user registration, authentication, and authorization mechanisms. It includes specialized government user management capabilities and access control systems, maintaining secure connections to the Ethereum blockchain for identity verification.

3. **Digital Identity Service:** Serving as the core component of the identity management ecosystem (Figure 1), this service handles the primary digital identity functionality. It manages identity token creation, maintains linkages between different identity components, and coordinates with other microservices to ensure seamless identity operations across the platform.

4. **Birth Certificate Service:** Integrated within the Application Layer (Figure 1), this service manages the complete lifecycle of digital birth certificates, including creation, storage, and verification processes. It maintains direct interfaces with blockchain smart contracts through the Ethereum Network layer, ensuring immutable record keeping and tamper-proof certificate storage.

5. **Scholar Curriculum Service:** Positioned alongside other microservices in the Application Layer (Figure 1), this service manages educational credentials and academic achievements. It provides verifiable academic record storage and retrieval capabilities, leveraging blockchain technology for credential authenticity and preventing fraudulent academic claims.

6. **Frontend Services:** As depicted in the Presentation Layer of Figure 1, four separate web applications provide comprehensive user interfaces for different system components. Each frontend service implements responsive design principles and maintains real-time blockchain interaction capabilities, ensuring seamless user experience across all identity management functions.

1.3 Network Architecture

The system architecture, as comprehensively shown in Figure 1, utilizes a custom Docker network configuration that encompasses the entire system infrastructure. This Docker Network Infrastructure serves as the foundational layer that contains both the Presentation Layer (four Frontend Services) and the Application Layer (API Gateway and four microservices), ensuring secure inter-service communication while maintaining strict service isolation.

Each service operates within a dedicated container environment with specific IP address assignments, enabling controlled network access and enhanced security protocols. The

Docker network configuration facilitates seamless communication between frontend applications and backend microservices through the API Gateway, while all services maintain secure connections to the external Ethereum Network for blockchain operations. This architectural approach ensures scalability, maintainability, and robust security across all system components.

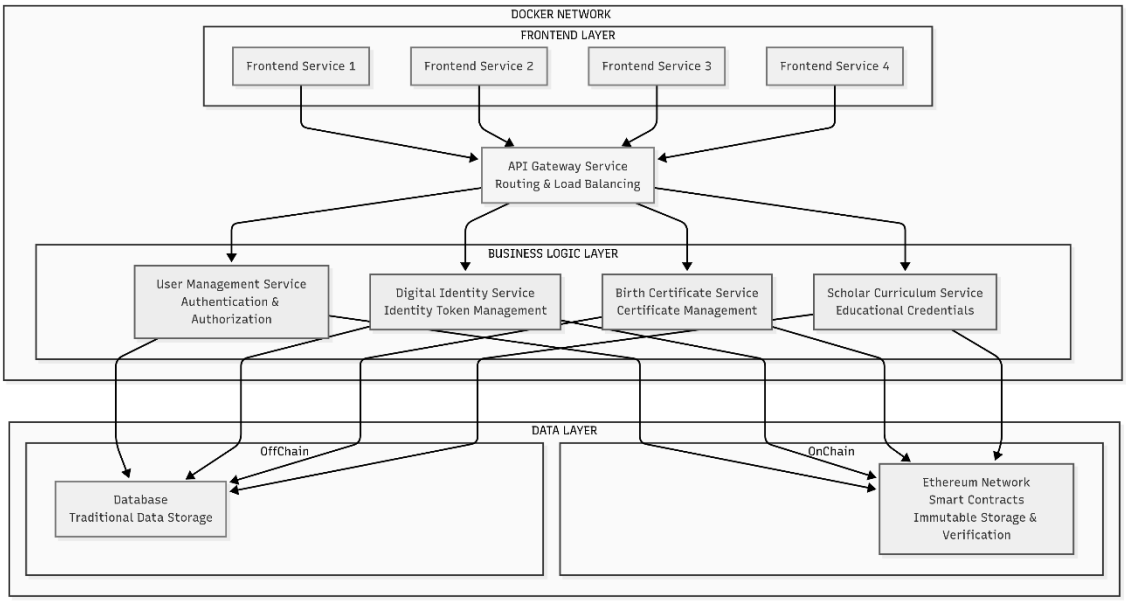


Figure 1

2. Technical Implementation

As illustrated in Figure 2, the ChainIDentity system employs a comprehensive three-layer architecture deployed within a Docker network infrastructure, ensuring secure inter-service communication and scalable deployment.

2.1 Data Layer Architecture

The data layer, as shown in Figure 2, implements a hybrid storage approach with both on-chain and off-chain components:

On-Chain Smart Contracts

The on-chain component implements four core smart contracts written in Solidity 0.8.23, each addressing specific identity management requirements:

1. Users Contract: Serves as the foundational user management contract, implementing role-based access control with government, administrator, and regular user classifications. The contract establishes the trust foundation for the entire system by managing user registration and verification processes.

2. DigitalIdentity Contract: Provides identity token management and linking capabilities, enabling users to connect multiple identity documents under a unified digital identity. The contract implements sophisticated verification mechanisms to ensure token authenticity and government certification status.

3. BirthCertificate Contract: Implements comprehensive birth certificate digitization with privacy-controlled access to sensitive information. The contract includes advanced features such as parental address linking and government-only modification capabilities.

4. ScholarCurriculum Contract: Manages educational credentials and achievements, providing institutions with tools to issue verifiable academic records.

Off-Chain Database Storage

The off-chain component utilizes MySQL databases for traditional data storage requirements, including user session management, encrypted password storage using Bcrypt, and non-critical application data that benefits from faster access times and lower transaction costs.

2.2 Business Logic Layer Implementation

Figure 2 demonstrates the business logic layer's four specialized microservices, each built using Node.js with Express.js framework and implementing RESTful API patterns:

Smart Contract Integration: All business services implement Web3.js integration for blockchain interaction, handling contract deployment, method invocation, and event monitoring. The implementation includes comprehensive error handling and transaction monitoring capabilities.

Database Integration: Services maintain dual connectivity - accessing MySQL databases for off-chain data storage while simultaneously interacting with smart contracts for immutable record keeping. This hybrid approach implements connection pooling and transaction management for optimal performance.

2.3 API Gateway Configuration

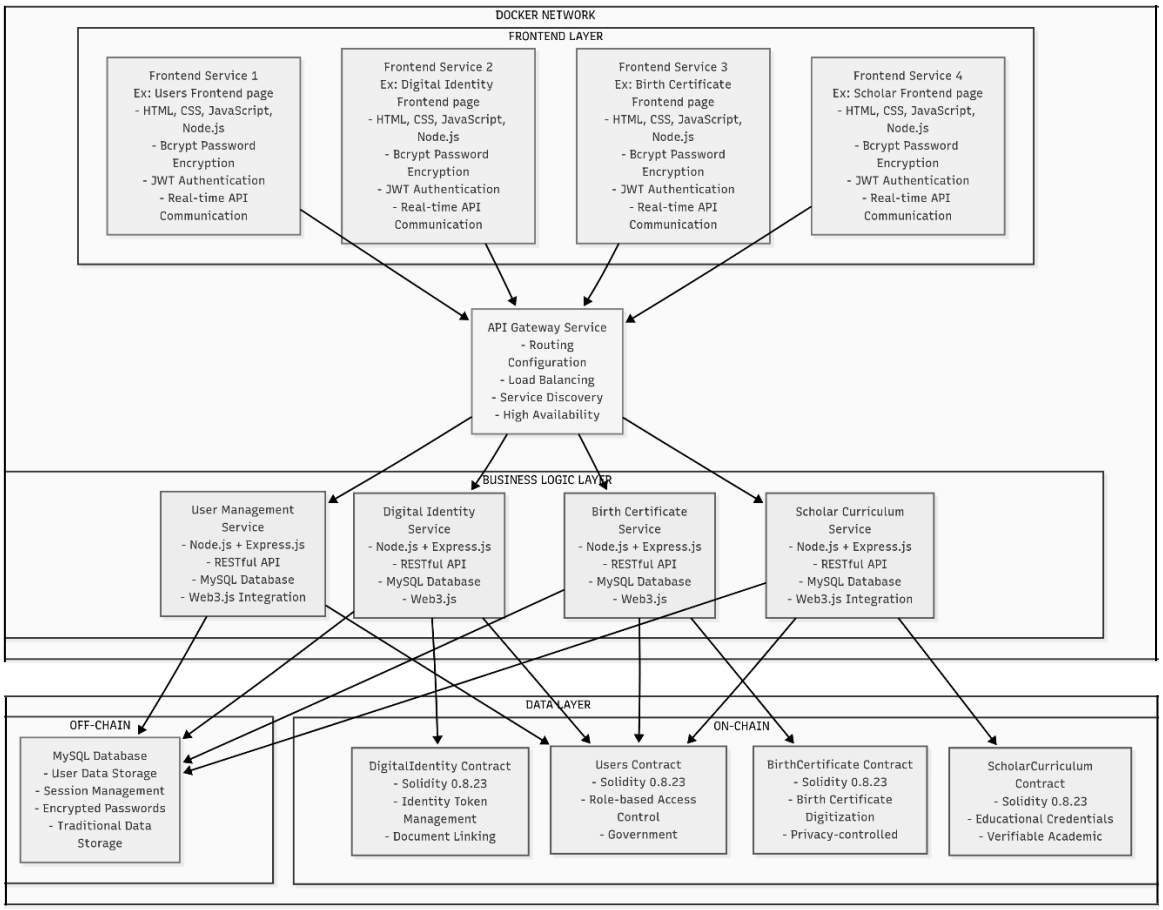
As depicted in Figure 2, the API Gateway operates as an independent service within the Docker network, implementing comprehensive routing configuration supporting both localhost and containerized deployment scenarios. The configuration includes load balancing, service discovery, and request forwarding capabilities that enable seamless service scaling and high availability across all four business logic services.

2.4 Frontend Layer Implementation

The frontend layer, illustrated in Figure 2, consists of four specialized web applications, each targeting specific identity management functions:

- **Users Frontend:** Handles user registration and authentication processes
- **Digital Identity Frontend**: Manages identity token creation and linking
- **Birth Certificate Frontend:** Provides birth certificate digitization interface
- **Scholar Frontend:** Facilitates educational credential management

Each frontend service implements modern web technologies including HTML, CSS, JavaScript, and Node.js, with Tailwind CSS for styling, Bcrypt for password encryption, and real-time API communication with JWT-based authentication. The services maintain consistent user experience across the ecosystem while providing specialized interfaces for their respective identity management functions.



3. Security Analysis

3.1 Blockchain Security

The smart contract implementation incorporates multiple security mechanisms:

Access Control: Role-based permissions ensure that only authorized users can perform specific operations. Government users have elevated privileges for identity verification.

Data Privacy: Sensitive personal information is protected through access modifiers and function-level permissions, ensuring that private data is only accessible to authorized parties.

Transaction Validation: Smart contracts implement comprehensive input validation and state verification to prevent unauthorized modifications and maintain data integrity.

3.2 Application Security

Authentication and Authorization: The system implements JWT-based authentication with bcrypt password hashing for secure user credential management.

Database Security: Connection pooling and parameterized queries prevent SQL injection attacks while maintaining optimal database performance.

Network Security: Container-based deployment with custom network configuration provides service isolation and controlled inter-service communication.

3.3 Cryptographic Verification

The system implements multiple layers of cryptographic verification:

Digital Signatures: All blockchain transactions are cryptographically signed, ensuring authenticity and non-repudiation.

Hash Verification: Document integrity is maintained through cryptographic hashing mechanisms embedded in smart contract functions. As an example hash verification happens when linking tokens - the system checks contract addresses, ownership, and government certification status before creating immutable records.

Certificate Validation: Government certification status is cryptographically verifiable through smart contract functions, like in the certificate validation using the user type verification where government users (type 0) have the authority to create and modify certificates.

4. Performance Evaluation

4.1 Platform Compatibility

The system has been successfully tested across multiple operating systems and hardware configurations, demonstrating robust cross-platform compatibility. Testing environments include various Windows versions, macOS, and Linux distributions, spanning different processor architectures and memory configurations.

4.2 Scalability Analysis

The microservices architecture enables horizontal scaling through container orchestration. Each service can be independently scaled based on demand, with the API Gateway providing load balancing and service discovery capabilities.

Container Resource Management: Docker-based deployment ensures consistent resource allocation and efficient utilization across different deployment environments.

Database Performance: MySQL integration with connection pooling optimizes database access patterns and maintains performance under varying load conditions.

4.3 Deployment Flexibility

The system supports multiple deployment scenarios:

Local Development: Complete local deployment using Docker Compose for development and testing purposes.

Cloud Deployment: AWS-compatible deployment configuration with RDS integration for production environments.

Hybrid Deployment: Flexible configuration supporting mixed on-premises and cloud deployment scenarios.

5. Use Cases and Applications

5.1 Government Identity Services

The system provides comprehensive support for government identity document digitization and verification. Birth certificate management enables secure issuance and verification of digital birth certificates with full audit trails and tamper-proof storage.

Citizen Registration: Government agencies can register citizens and issue digital identity tokens with cryptographic verification capabilities.

Document Verification: Third parties can verify identity document authenticity without accessing sensitive personal information, maintaining privacy while ensuring authenticity.

5.2 Educational Credential Management

The scholar curriculum component enables educational institutions to issue verifiable academic credentials that cannot be forged or falsified. Students maintain control over their academic records while enabling easy verification by employers and other institutions.

Degree Verification: Employers can instantly verify educational credentials without contacting issuing institutions, reducing verification time and costs.

Academic Record Portability: Students can transfer verified academic records between institutions without complex verification processes.

5.3 Enterprise Identity Integration

The system's API-first design enables integration with existing enterprise identity management systems, providing blockchain-based verification capabilities while maintaining compatibility with legacy systems.

6. Deployment and Operations

6.1 Automated Deployment

The system includes automated deployment scripts for both local and cloud environments. These scripts handle container orchestration, database initialization, and service configuration, reducing deployment complexity and potential configuration errors.

6.2 Monitoring and Health Checks

Built-in health check endpoints enable continuous monitoring of service availability and performance. The monitoring system provides real-time status information for all system components, enabling proactive maintenance and issue resolution.

6.3 Database Management

The system implements comprehensive database management procedures including table creation, user management, and data migration capabilities. Automated database initialization ensures consistent deployment across different environments.

7. Future Work

7.1 Interoperability Enhancement

Future development will focus on implementing standardized identity protocols such as W3C Decentralized Identifiers (DIDs) and Verifiable Credentials to enhance interoperability with other identity systems and blockchain networks.

7.2 Privacy Enhancement

Implementation of advanced privacy-preserving technologies such as zero-knowledge proofs and selective disclosure mechanisms will enable more granular privacy control while maintaining verification capabilities.

7.3 Mobile Integration

Development of mobile applications with biometric authentication and offline verification capabilities will extend the system's accessibility and usability for end users.

7.4 Advanced Analytics

Integration of blockchain analytics and user behavior monitoring will provide insights into system usage patterns and enable proactive security monitoring.

7.5 Regulatory Compliance

Enhanced compliance features for various international identity regulations and standards will enable broader adoption across different jurisdictions and regulatory environments.

8. Software Availability

8.1 Repository Access

The complete ChainIDentity source code, documentation, and deployment instructions are available in the public GitHub repository:

GitHub Repository: https://github.com/UPTokenizing/digitalIdentity

8.2 QR Code Access

ChainIDentity GitHub Repository



`github.com/UPTokenizing/digitalIdentity`

Scan the QR code to access the repository on mobile devices

8.3 System Requirements

The ChainIDentity platform requires the following environment setup for deployment:

Prerequisites:

- **Docker & Docker Compose:** For containerized microservices deployment
- **Node.js:** Version 14.x or higher for backend services
- **Ganache CLI:** Local blockchain development environment
- **MySQL Server:** Database management system
- **Web3.js**: Ethereum blockchain interaction library

Port Configuration:

- **Users Frontend Service**: http://localhost:5512
- **Digital Identity Service**: http://localhost:5511
- **Birth Certificate Service**: http://localhost:5510
- **Scholar Curriculum Service**: http://localhost:5513

8.4 Installation and Deployment

Comprehensive installation instructions, including environment configuration, Docker container setup, smart contract deployment procedures, and database initialization scripts, are available in the repository's README.md file. The documentation provides step-by-step guidance for both development and production environments.

8.5 License and Contribution

The project is available under open-source licensing terms as specified in the repository. Contributors are welcome to submit issues, feature requests, and pull requests following the established contribution guidelines.

8.6 User Manual Summary

The ChainIDentity User Manual describes a decentralized digital identity management system built on blockchain technology that enables tokenization of entities and physical/digital objects.

Key Features:

- **4-Service Web Architecture**: Users (port 5512), Digital Identity (5511), Birth Certificate (5510), Scholar Curriculum (5513)
- **Role-Based Access Control:** Government, Administrator, Regular User, Educational Administrator, Guest
- **Smart Contract Integration**: 4 Solidity contracts managing users, identities, certificates, and academic records**.**

User Workflow:

- **System Initialization:** Mandatory government user registration at /registergovernment
- **User Management**: Government-controlled user registration and administration

Specialized Services:

- **Digital Identity**: Personal dashboard and token linking capabilities
- **Birth Certificates**: Digitization with privacy-controlled access
- **Academic Records**: Educational achievement tokenization

Core Functionalities:

- Public search by token address or email
- Physical/digital object linking to digital identities
- Role-based permission system
- Decentralized ownership verification
- Blockchain integration for immutable record keeping

The system is designed for governmental and institutional use cases, providing a trusted foundation for verifiable digital identity management with comprehensive audit trails and privacy controls.

Bibliography

[1] A.-E. Panait, R. F. Olimid, and A. Stefanescu, “Identity Management on Blockchain – Privacy and Security Aspects,” arXiv preprint arXiv:2004.13107, 2020. [Online]. Available: https://acad.ro/sectii2002/proceedings/doc2020-1/06-Panait.pdf

[2] D. Tran, “Blockchain Use Cases: Digital Identity,” FullstackO, Nov. 2022. [Online]. Available: https://www.fullstacko.com/blog/blockchain-use-cases-digital-identity

[3] D. Zhang, “Challenging the Centralized Status Quo with Decentralized Identity Innovations,” CMU Blockchain Group – Medium, May 2021. [Online]. Available: https://medium.com/cmu-blockchain-group/challenging-the-centralized-status-quo-with-decentralized-identity-innovations-4ffdace5ea4c

[4] A. Mühle, A. Grüner, T. Gayvoronskaya, and C. Meinel, “A survey on essential components of a self-sovereign identity,” Computer Science Review, vol. 30, pp. 1–29, 2018. [Online]. Available: https://doi.org/10.1016/j.cosrev.2018.10.002

[5] J. Chen, X. Shi, Y. Liu, K. Zhang, and Y. Zhang, “A survey on blockchain-based identity management systems for the Internet of Things,” Computers & Security, vol. 108, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0167404820303230